



IMD1116 COMPUTAÇÃO DE ALTO DESEMPENHO

Prof. Samuel Xavier
de Souza

HISTÓRICO E EVOLUÇÃO DA PROGRAMAÇÃO

- Objetivos desse tópico
 - Reconhecer a necessidade de maior conhecimento do hardware para alcançar alto desempenho [Tarefas 1 e 2]
 - Reconhecer a necessidade da programação paralela para alcançar alto desempenho [Tarefa 3]

HISTÓRICO E EVOLUÇÃO DA PROGRAMAÇÃO

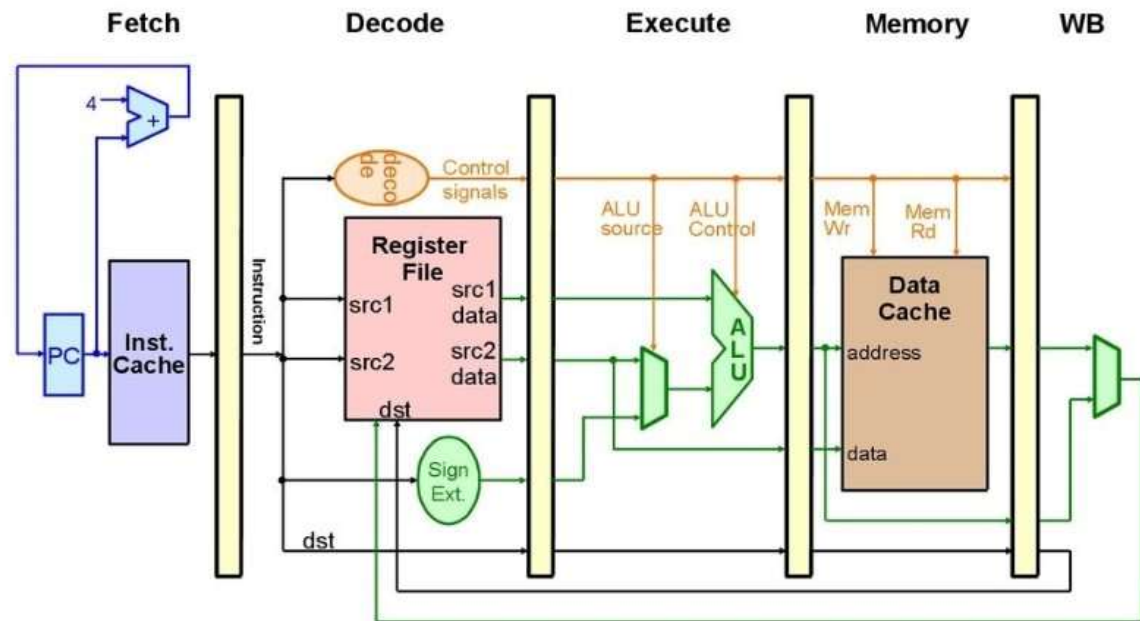
Tarefa 1:

Implemente duas versões da multiplicação de matriz por vetor ($M \times V$) em C: uma com acesso à matriz por linhas (linha externa, coluna interna) e outra por colunas (coluna externa, linha interna). Meça o tempo de execução de cada versão com uma função apropriada e execute testes com diferentes tamanhos de matriz. Identifique, a partir de que tamanho, os tempos passam a divergir significativamente e explique por que isso ocorre, relacionando suas observações ao uso da memória cache e ao padrão de acesso à memória.

- O gargalo de von Neumman e a memória cache
 - Localidade temporal e espacial
 - Row-major vs. column-major

HISTÓRICO E EVOLUÇÃO DA PROGRAMAÇÃO

- ILP – Paralelismo ao nível de instrução
 - Pipelining,
 - Vetorização



HISTÓRICO E EVOLUÇÃO DA PROGRAMAÇÃO

Instrução	1				2			
Fetch								
Decode								
Execute								
Write								
Ciclo	1	2	3	4	5	6	7	8

Instrução	1	2			
Fetch					
Decode					
Execute					
Write					
Ciclo	1	2	3	4	5

HISTÓRICO E EVOLUÇÃO DA PROGRAMAÇÃO

Tarefa 2:

Implemente três laços em C para investigar os efeitos do paralelismo ao nível de instrução (ILP): 1) inicialize um vetor com um cálculo simples; 2) some seus elementos de forma acumulativa, criando dependência entre as iterações; e 3) quebre essa dependência utilizando múltiplas variáveis. Compare o tempo de execução das versões compiladas com diferentes níveis de otimização (O0, O2, O3) e analise como o estilo do código e as dependências influenciam o desempenho.

Instrução	1				2			
Fetch								
Decode								
Execute								
Write								
Ciclo	1	2	3	4	5	6	7	8

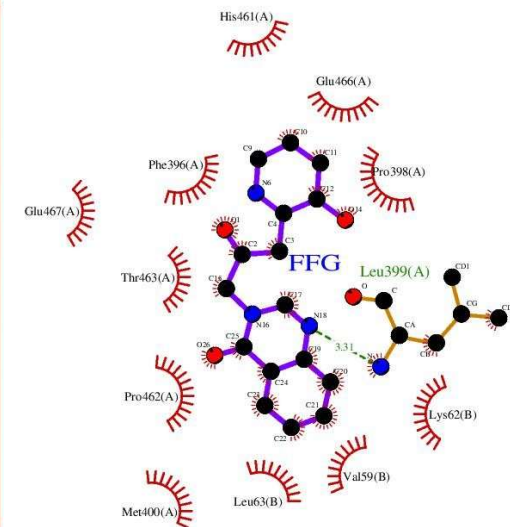
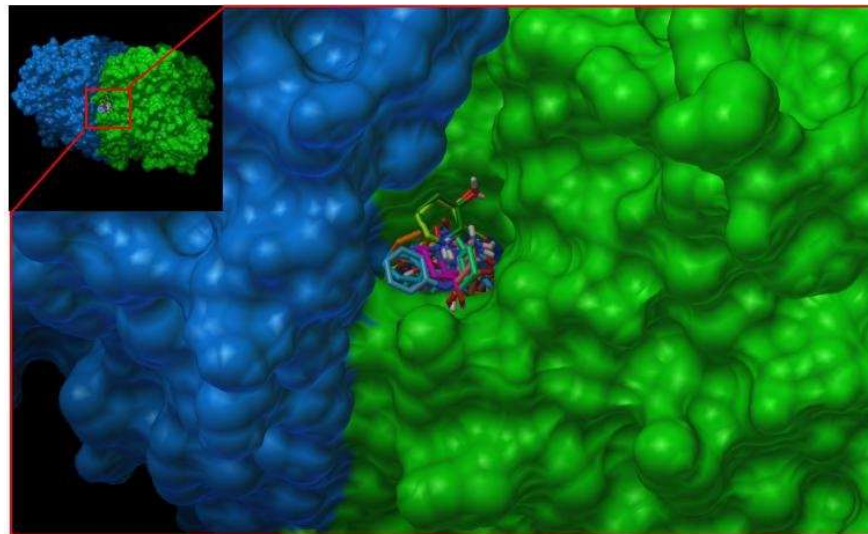
Instrução	1	2			
Fetch					
Decode					
Execute					
Write					
Ciclo	1	2	3	4	5

HISTÓRICO E EVOLUÇÃO DA PROGRAMAÇÃO

- Por que algumas aplicações requerem maior poder computacional continuamente?

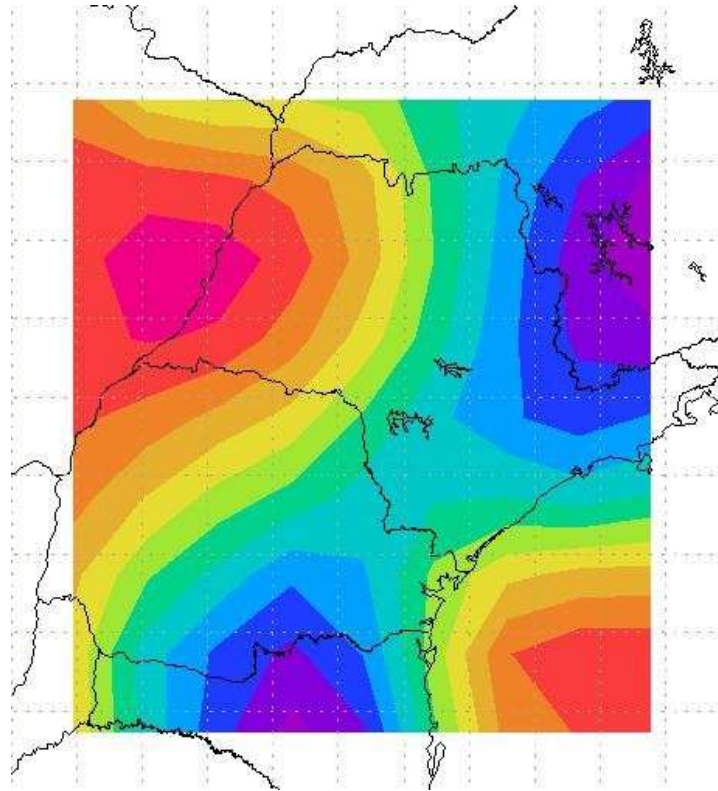
HISTÓRICO E EVOLUÇÃO DA PROGRAMAÇÃO

- Simulações científicas para descoberta de novas drogas



HISTÓRICO E EVOLUÇÃO DA PROGRAMAÇÃO

- Modelagem de sistemas de previsão de tempo



HISTÓRICO E EVOLUÇÃO DA PROGRAMAÇÃO

- Treinamentos de modelos de IA

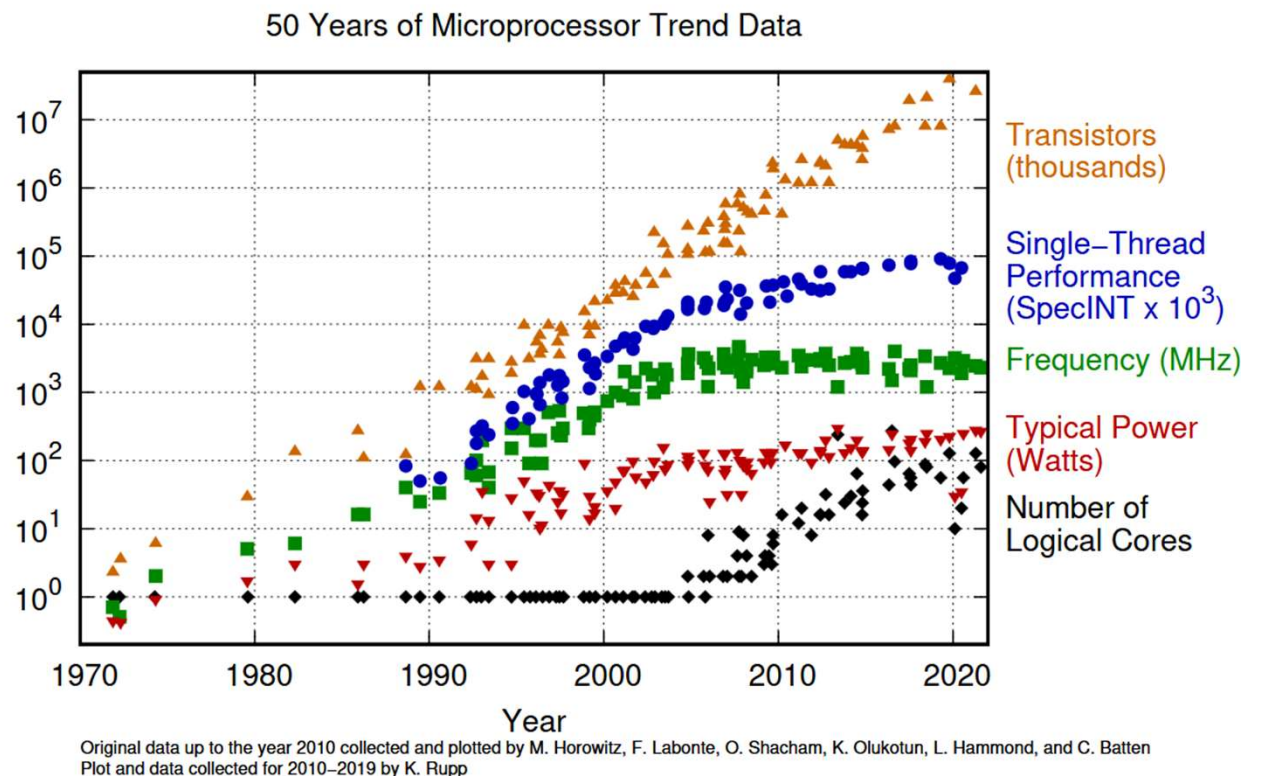


HISTÓRICO E EVOLUÇÃO DA PROGRAMAÇÃO

Tarefa 3:

Implemente um programa em C que calcule uma aproximação de π por meio de uma série matemática, variando o número de iterações e medindo o tempo de execução. Compare os valores obtidos com o valor real de π e analise como a acurácia melhora com mais processamento. Reflita sobre como esse comportamento se repete em aplicações reais que exigem resultados cada vez mais precisos, como simulações físicas e inteligência artificial.

- As limitações físicas da fabricação de processadores sequenciais mais rápidos



PROGRAMAS MULTITAREFAS

- Objetivos desse tópico
 - Reconhecer a necessidade de maior conhecimento do software para alcançar alto desempenho
 - Reconhecer a maior dificuldade de se programar em paralelo

PROGRAMAS MULTITAREFAS

Tarefa 4:

Implemente dois programas paralelos em C com OpenMP: um limitado por memória, com somas simples em vetores, e outro limitado por CPU, com cálculos matemáticos intensivos. Paralelize com `#pragma omp parallel for` e meça o tempo de execução variando o número de threads. Analise quando o desempenho melhora, estabiliza ou piora, e reflita sobre como o multithreading de hardware pode ajudar em programas memory-bound, mas atrapalhar em programas compute-bound pela competição por recursos.

- Quais programas são limitados pela memória (gargalo de von Neuman) e quais não são?
 - Como o multi-thread de hardware pode ajudar (ou atrapalhar)

```
$ gcc -fopenmp meuprog.c -o meuprog
```

```
$ export OMP_NUM_THREADS=2; time meuprog
```

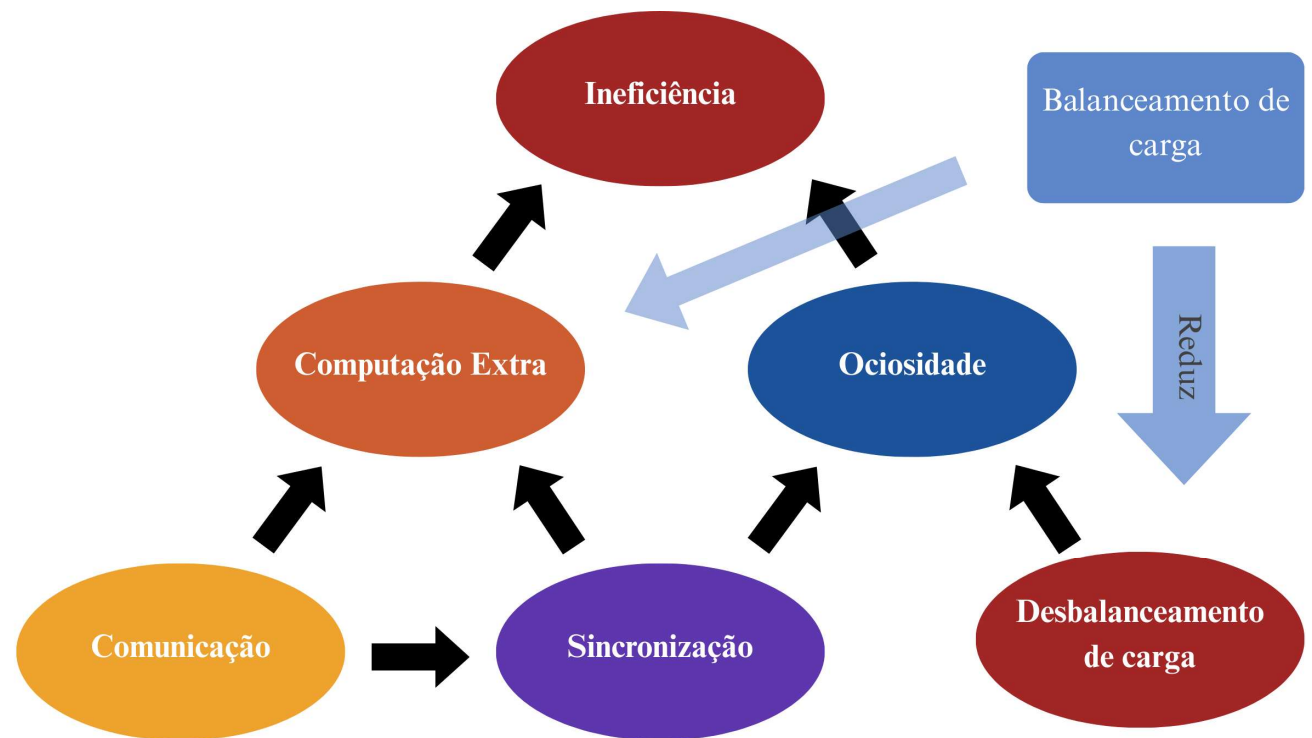
PROGRAMAS MULTITAREFAS

- Por que é mais difícil programar em paralelo?
 - Sincronização, comunicação e equilíbrio de carga



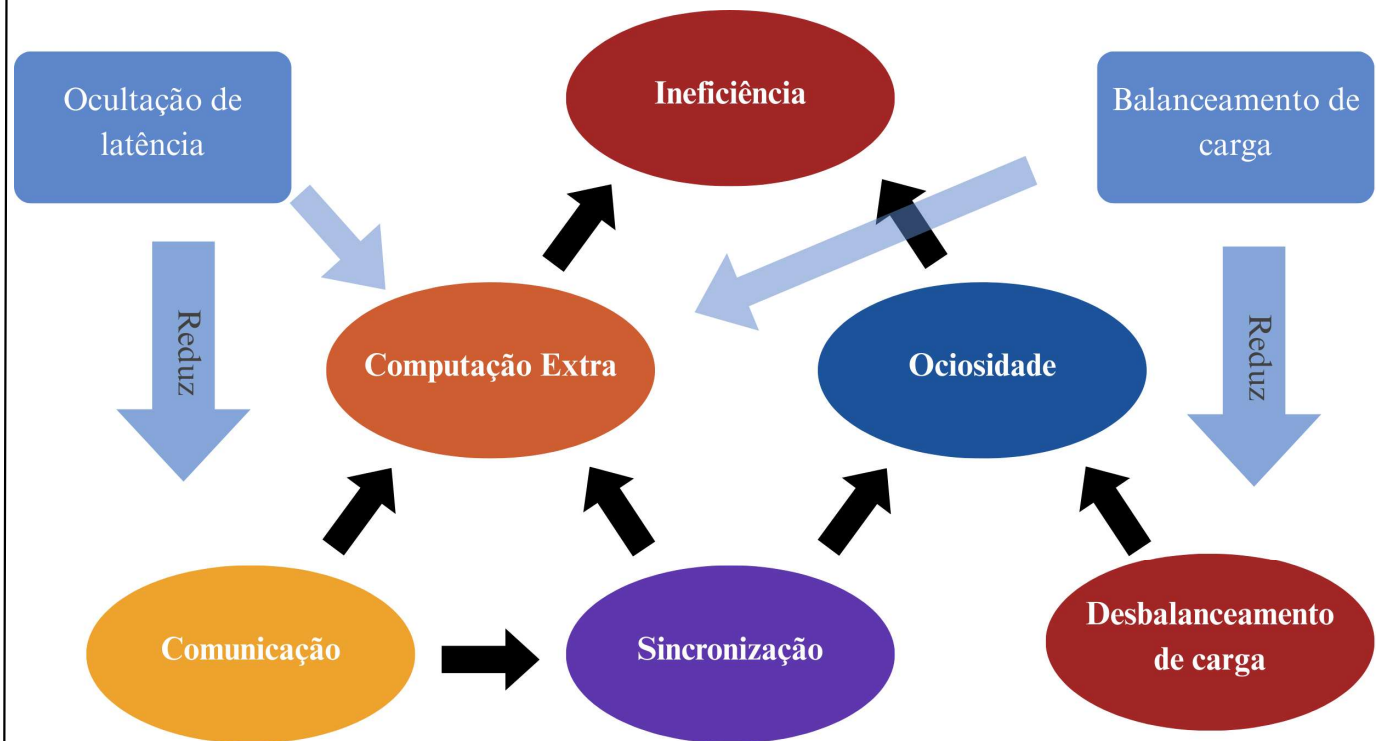
PROGRAMAS MULTITAREFAS

- Por que é mais difícil programar em paralelo?
 - Sincronização, comunicação e equilíbrio de carga



PROGRAMAS MULTITAREFAS

- Por que é mais difícil programar em paralelo?
 - Sincronização, comunicação e equilíbrio de carga



PROGRAMAS MULTITAREFAS

Tarefa 5:

Implemente um programa em C que conte quantos números primos existem entre 2 e um valor máximo n . depois, paralelize o laço principal usando a diretiva `#pragma omp parallel for` sem alterar a lógica original. compare o tempo de execução e os resultados das versões sequencial e paralela. observe possíveis diferenças no resultado e no desempenho, e reflita sobre os desafios iniciais da programação paralela, como correção e distribuição de carga.

- Por que é mais difícil programar em paralelo?
 - Sincronização, comunicação e equilíbrio de carga

```
$ gcc -fopenmp meuprog.c -o meuprog
```

PROGRAMAÇÃO EM MEMÓRIA COMPARTILHADA

- Objetivos desse tópico
 - Conhecer as ferramentas básicas de programação em memória compartilhada

PROGRAMAÇÃO EM MEMÓRIA COMPARTILHADA

- Parallel e parallel for
- Escopo de variáveis
- Condição de corrida e regiões críticas

PROGRAMAÇÃO EM MEMÓRIA COMPARTILHADA

Tarefa 6:

Implemente em C a estimativa estocástica de π . Paralelize com `#pragma omp parallel for` e explique o resultado incorreto. Corrija a condição de corrida utilizando o `#pragma omp critical` e reestruturando com `#pragma omp parallel` seguido de `#pragma omp for` e aplicando as cláusulas `private`, `firstprivate`, `lastprivate` e `shared`. Teste diferentes combinações e explique como cada cláusula afeta o comportamento do programa. Comente também como a cláusula `default(none)` pode ajudar a tornar o escopo mais claro em programas complexos.

- `Parallel` e `parallel for`
- Escopo de variáveis
- Condição de corrida e regiões críticas

PROGRAMAÇÃO EM MEMÓRIA COMPARTILHADA

- `#pragma omp single`
- `#pragma omp master`
- `#pragma omp barrier`
- Cláusula `nowait`
- Modelo de tarefas
 - `#pragma omp task`
 - `#pragma omp taskwait`

PROGRAMAÇÃO EM MEMÓRIA COMPARTILHADA

Tarefa 7:

Implemente um programa em C que cria uma lista encadeada com nós, cada um, contendo o nome de um arquivo fictício. Dentro de uma região paralela, percorra a lista e crie uma tarefa com `#pragma omp task` para processar cada nó. Cada tarefa deve imprimir o nome do arquivo e o identificador da thread que a executou. Após executar o programa, reflita: todos os nós foram processados? Algum foi processado mais de uma vez ou ignorado? O comportamento muda entre execuções? Como garantir que cada nó seja processado uma única vez e por apenas uma tarefa?

- `#pragma omp single`
- `#pragma omp master`
- `#pragma omp barrier`
- Cláusula `nowait`
- Modelo de tarefas
 - `#pragma omp task`
 - `#pragma omp taskwait`